

Who Can Turn Off Your Ethereum App?

A Full-Stack Walk-Away Test

James Carnley

Ethereum File System



Building Cypherpunk Apps

Why make an app you can walk away from?

What happens when an app disappears?

- Users lose their work and trust
- Other developers can’t rely on that functionality
- Communities splinter

Software worth building on

For people

Keep years of useful work

Choose better tools

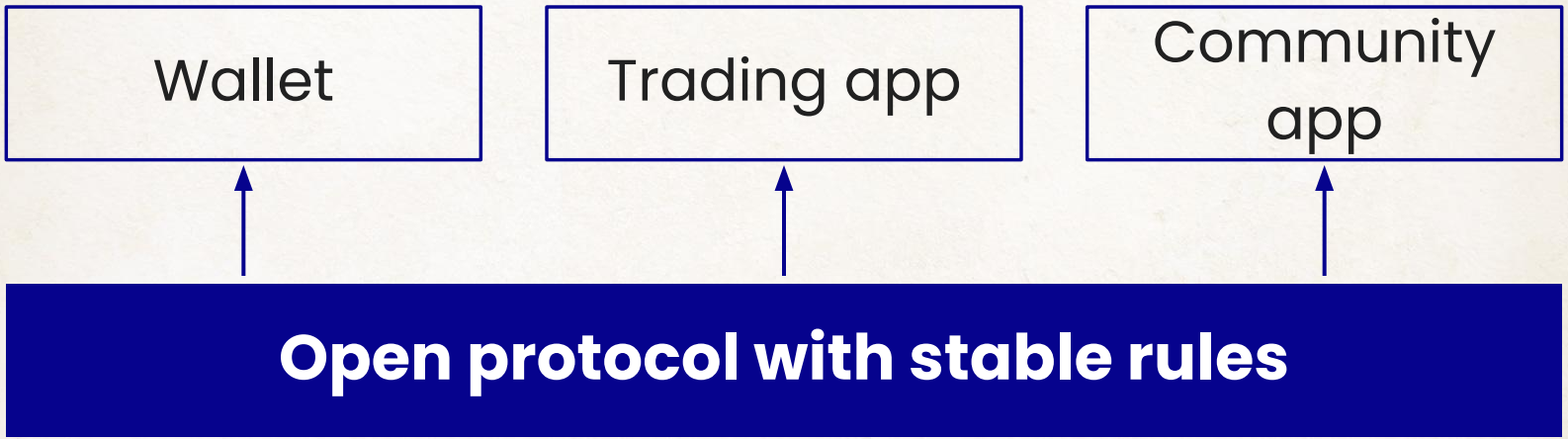
For builders

Reuse dependable capabilities

Build bigger things together

Hyperstructures: shared foundations

Public protocols that work without their creators



Each new app adds value to the shared foundation

A walk-away test

The team disappears

Can users keep working?

Can others run the services?

Control becomes hostile

Can users keep their work?

Can they keep the rules they chose?

The application stack

Interface

- Names and discovery
- Runnable software
- Accounts and recovery

Data

- Storage and formats
- State and indexes
- Background work

Rules

- Contracts and governance
- Network and settlement



Interface

Finding and using the app

Find a saved release

Run it without your servers

Recover access without your help

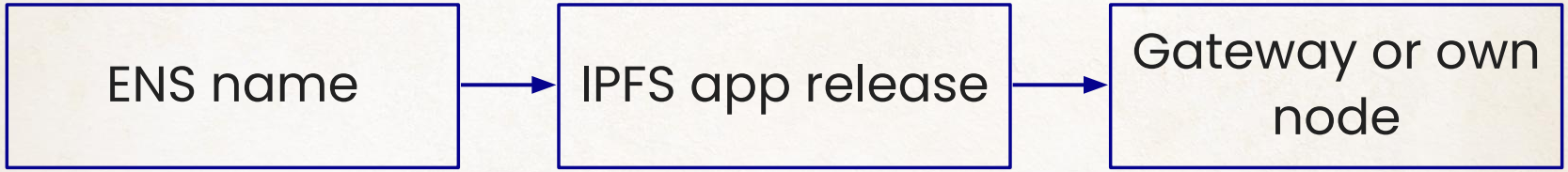
Can people find and use your app?

More than one way into the app

DNS + HTTP



ENS + IPFS



Save the release and the instructions to open it

A release others can run

- Keep a complete runnable release
- Include the code it loads later
- Publish source and build instructions
- Try a fresh-machine rebuild

Access people can recover

- Give users a recovery route
- Safe example: two of three signers
- Back up decryption keys for private files
- Practice a lost-device recovery



Portable Data

Continuing the work

Is the data portable?

Keep the files and their meaning

Use them through another app

Let others run the supporting services

Data other apps can understand

EAS

Attestation schemas

Who said what

EFS

Shared file paths

Provenance and references

Nostr

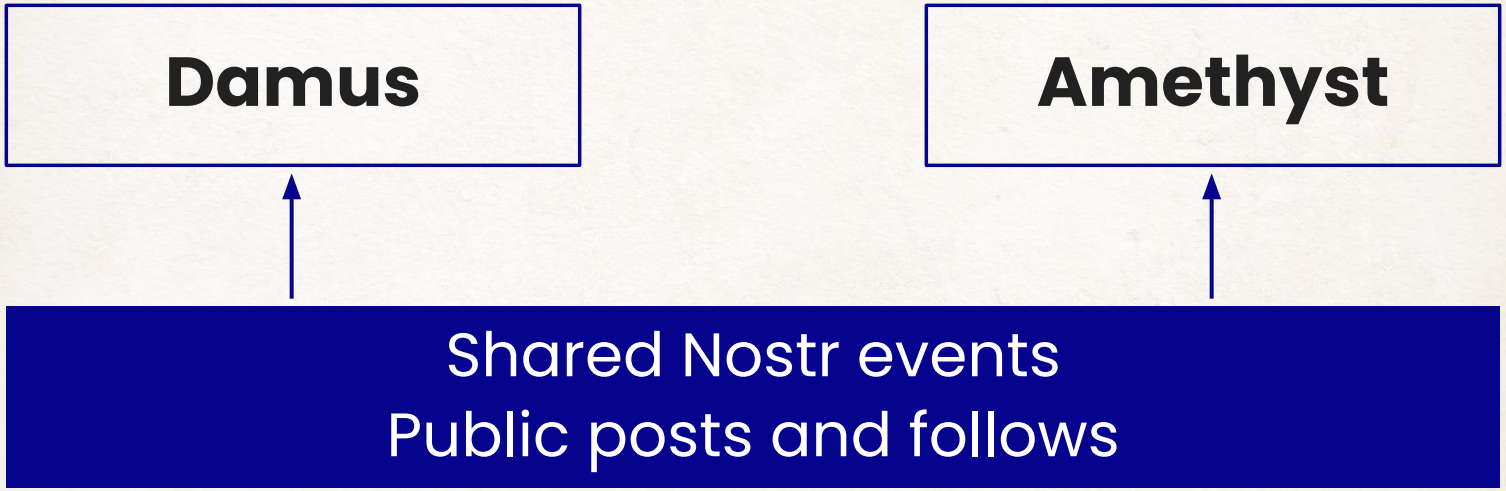
Signed social events

Public posts and follows

Publish the structure so another app can use the same records

Two apps, one social identity

Your key lets you participate through either app

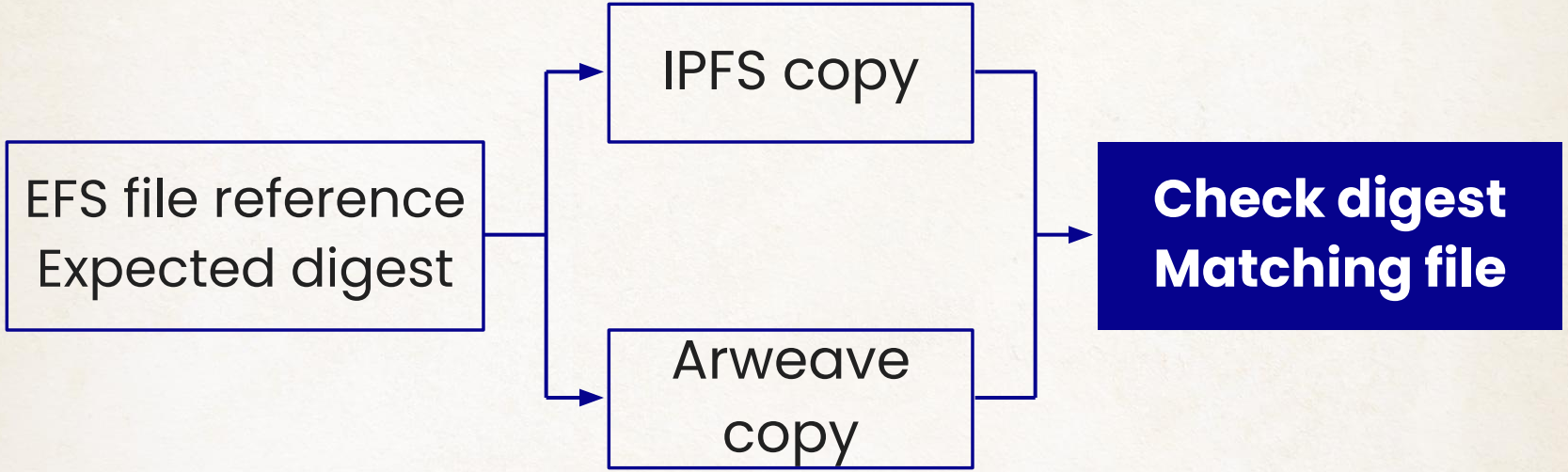


Choose a different interface. Keep participating.

Data people can keep using

- Retain copies with independent operators
- Use documented formats and identifiers
- Protect private data and recover its keys
- Restore a real piece of work from a backup

EFS: a file you can verify



Keep the reference and choose where to retrieve the file

A dependable view of the data

- Let users choose an RPC or their own node
- Use verified reads where supported
- Show how recent the data is
- Publish what another indexer needs to rebuild

Work another operator can do

- Publish the worker code and required inputs
- Make the job open to another operator
- Give someone a reason to run it
- Example: PoolTogether prize claimers



Rules

Keeping dependable rules

Stable core contracts

User-chosen app upgrades

Clear rules for shared governance

Who chooses the changes?

Immutable contracts

A stable core

Keep the contract code fixed

Let other apps build on it

Room to improve

Deploy a new version

Let users choose to migrate

Example: Uniswap v2 core

The network beneath the app

Ethereum and L1

Keep a working node route

Retain the history you need

Understand inclusion and finality

L2 and settlement

Know who orders and proves

Find the required chain data

Practice the recovery route

A community can carry it forward

- Hic et Nunc community mirrors kept an entrance open
- Teia continued the community's work
- Share code, retained data and operating instructions
- Plan who can run it and pay for it



Putting this into practice

A walk-away test for your app

Your walk-away checklist

A user can:

Run a saved app release

Recover access and keys

Restore usable data

Choose another client

Another operator can:

Switch RPCs; rebuild views

Run workers; keep copies

Inspect admin powers

Use network recovery

Try it without your team. Improve one dependency.

Our work can outlive us


Thank you!
Let’s build reliable and long-lasting apps together
James Carnley
Ethereum File System
efs.eth.limo
JamesCarnley@gmail.com
@JamesCarnley